SYSTEMS ENGINEERING PORTFOLIO · LINUX / FOSS

Eighteen programs, and something outside each one that decides if it is right.

Every project here is built against an **external oracle** — gcc and clang, the Linux kernel, `dash`, GNU tar, readelf, SQLite, Python's own regex engine. Not one of them is the judge of its own correctness. That constraint is the whole method, and it is why **10 of the 18 defects recorded here were in the test harness rather than in the code**.

[your name — rebuild with NAME="..." to set it]

19	304	20,030	11,141	18/18
REPOSITORIES	COMMITS	LINES OF SOURCE	LINES OF TESTS	SUITES GREEN

Every figure in this document is generated from the repositories themselves: commit counts from `git`, line counts from the files, test counts from running each suite. Nothing is transcribed by hand.

The method

A test you wrote, checking code you wrote, against a number you worked out yourself, proves that you were consistent. It does not prove you were right. **Every project here is measured against an authority it does not control.**



How the eighteen compose. Each arrow is a dependency recorded in a PLAN.md. The three process tools in the bottom lane act on all the others: **unrepro** found a real reproducibility defect in eight of the eighteen, **commitq** gates every commit in all of them, and **srcdiff** audits published artifacts against the repositories they claim to come from.

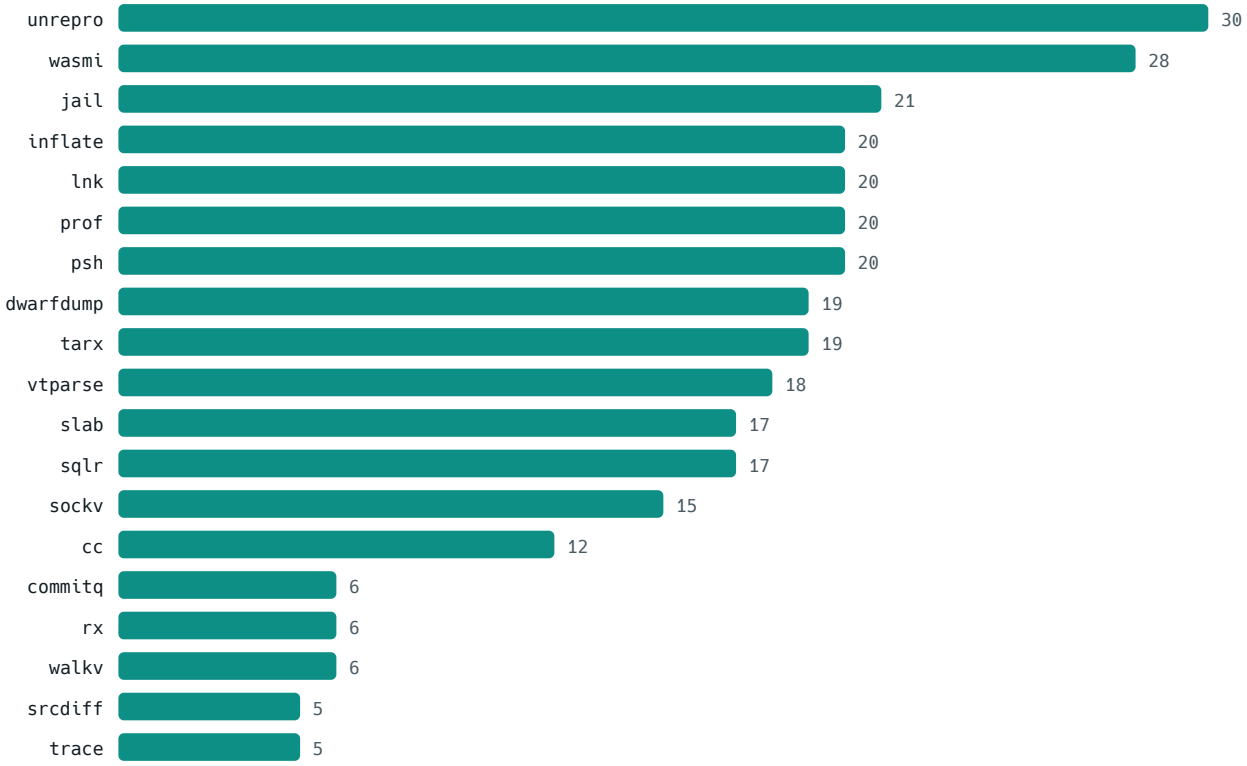
WHY THIS MATTERS MORE THAN IT SOUNDS

The first run of almost every harness in this project passed. The useful question is never “do my tests pass” but **“have I watched this harness fail?”** Where that question got asked, it usually turned out the answer was no — and the suite had been reporting success because it was silently doing nothing.



Defects recorded in commit messages and READMEs across all eighteen repositories. Counted once per root cause.

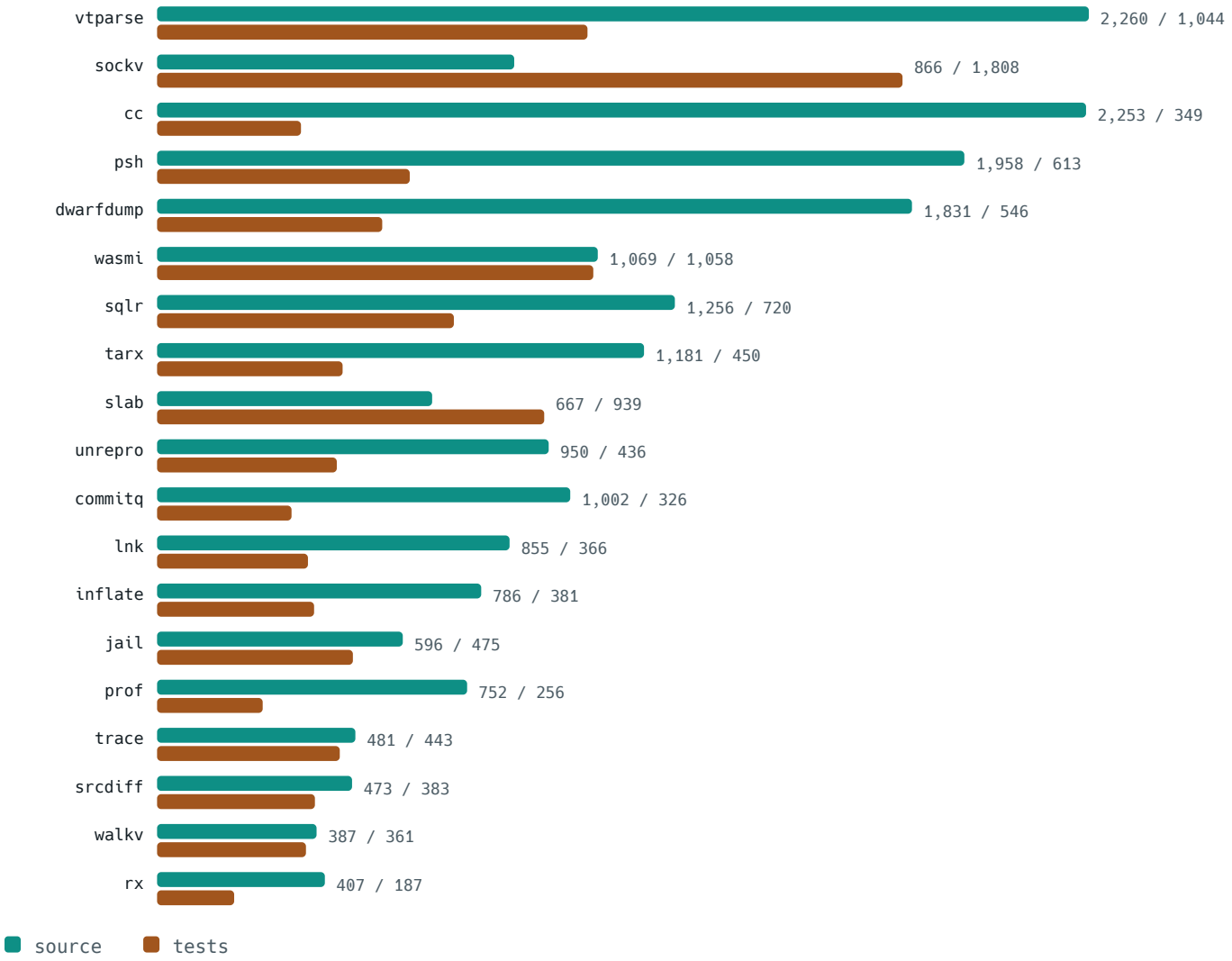
Scale



Commits per repository. Granularity is deliberate: every commit builds, verified by removing build artifacts and rebuilding each one in turn.

Source against tests

11,141 lines of tests to 20,030 of source. The ratio is the point: in several of these the harness **is** the project, and the program under test is scaffolding for it.



Lines of source and lines of test per repository, sorted by total. Labels read source / tests.

A NUMBER WORTH DISTRUSTING

Test *counts* are not comparable between these repos and are deliberately not charted together. `wasmi`'s 11,833 are generated arithmetic cases; `jail`'s 50 are hand-written assertions against kernel behaviour, and each one took longer to get right than a thousand of the former. Plotting them on one axis would imply a comparison that does not exist.

REPO	WHAT THE SUITE ACTUALLY REPORTS	PLAN
`cc`	168 passed, 0 failed	7/22
`commitq`	57 passed, 0 failed	5/16
`dwarfdump`	65 passed, 0 failed	17/27

REPO	WHAT THE SUITE ACTUALLY REPORTS	PLAN
inflate	159 passed, 0 failed	20/30
jail	60 passed, 0 failed, 0 skipped	14/23
lnk	29 passed, 0 failed	18/31
prof	15 passed, 0 failed, 0 skipped	18/30
psh	185 agree, 0 disagree, 0 skipped	19/30
rx	3000 agree, 0 disagree, 0 skipped (seed=0)	5/16
slab	30 passed, 0 failed	17/29
sockv	82 passed, 0 failed, 1 skipped	11/21
sqlr	114 agree, 0 disagree	17/28
srcdiff	25 passed, 0 failed	6/17
tarx	118 passed, 0 failed	7/19
trace	23 passed, 0 failed	11/20
unrepro	101 passed, 0 failed	12/24
vtparse	98 passed, 0 failed	9/20
walkv	163 scenarios checked, 0 failed	10/18
wasmi	11833 passed, 0 failed	25/35

The repositories

Grouped by what kind of thing they have to survive. Each card names the authority that decides whether it is correct, and the one property that, if violated, means the project failed.

Toolchain

Programs that build or run other programs.

wasmi — WebAssembly interpreter

28 commits · 1,069 lines · 1,058 test lines

wasmi → Expectations computed independently in Python

HARD INVARIANT	A malformed module is rejected or traps, never corrupts the host
RESULT	11833 passed, 0 failed
PLAN	25 done, 10 queued

lnk — Static ELF linker

20 commits · 855 lines · 366 test lines

lnk → The kernel runs the binary, or it does not

HARD INVARIANT	Every executable emitted runs, or we refused to emit it
RESULT	29 passed, 0 failed
PLAN	18 done, 13 queued

psh — POSIX shell subset

20 commits · 1,958 lines · 613 test lines

psh → dash: stdout AND exit status

HARD INVARIANT	Identical stdout and status to dash on every corpus script
RESULT	185 agree, 0 disagree, 0 skipped
PLAN	19 done, 11 queued

cc — C compiler for x86-64

12 commits · 2,253 lines · 349 test lines

cc → gcc AND clang must agree on every case

HARD INVARIANT	Self-host fixpoint: cmp cc2 cc3
RESULT	168 passed, 0 failed
PLAN	7 done, 15 queued

Formats & parsers

Programs that read a specification exactly.

inflate — DEFLATE / gzip codec

20 commits · 786 lines · 381 test lines

inflate → gzip -d accepts our output, both directions

HARD INVARIANT	gzip accepts all we emit; we decode all it emits
RESULT	159 passed, 0 failed
PLAN	20 done, 10 queued

dwarfdump — DWARF debug-info parser

19 commits · 1,831 lines · 546 test lines

dwarfdump → readelf --debug-dump

HARD INVARIANT	Every fact printed agrees with readelf
RESULT	65 passed, 0 failed
PLAN	17 done, 10 queued

tarx — tar reader and writer

19 commits · 1,181 lines · 450 test lines

tarx → GNU tar, bidirectional interop

HARD INVARIANT	GNU tar agrees both ways, and no member escapes the destination
RESULT	118 passed, 0 failed
PLAN	7 done, 12 queued

vtparse — VT500 parser and wcwidth

18 commits · 2,260 lines · 1,044 test lines

vtparse → The published DEC state table, all 3584 pairs

HARD INVARIANT	Never allocates, never reads past its byte, matches the table
RESULT	98 passed, 0 failed
PLAN	9 done, 11 queued

sqlr — SQLite file-format reader

17 commits · 1,256 lines · 720 test lines

sqlr → libsqlite3 itself

HARD INVARIANT	Byte-identical output to SQLite; malformed input errors, never crashes
RESULT	114 agree, 0 disagree
PLAN	17 done, 11 queued

Systems

Programs that must survive the machine misbehaving.

jail — Landlock sandbox

21 commits · 596 lines · 475 test lines

jail → The kernel: denied means EACCES, verified

HARD INVARIANT — An ungranted path is never reachable; the child cannot relax itself

RESULT — `60 passed, 0 failed, 0 skipped`

PLAN — 14 done, 9 queued

slab — LD_PRELOAD malloc replacement

17 commits · 667 lines · 939 test lines

slab → Real programs must produce identical output

HARD INVARIANT — Two live allocations never overlap; every pointer aligned

RESULT — `30 passed, 0 failed`

PLAN — 17 done, 12 queued

sockv — netlink socket enumerator

15 commits · 866 lines · 1,808 test lines

sockv → ss from iproute2, both directions

HARD INVARIANT — Every socket the kernel reports is listed once with every field as given

RESULT — `82 passed, 0 failed, 1 skipped`

PLAN — 11 done, 10 queued

rx — Linear-time regex engine

6 commits · 407 lines · 187 test lines

rx → Python's re module

HARD INVARIANT — A pc enters a thread list at most once per input position

RESULT — `3000 agree, 0 disagree, 0 skipped (seed=0)`

PLAN — 5 done, 11 queued

walkv — Write-ahead-log key-value store

6 commits · 387 lines · 361 test lines

walkv → A crash injected at every I/O boundary

HARD INVARIANT — Recovered state is a prefix with no holes

RESULT — `163 scenarios checked, 0 failed`

PLAN — 10 done, 8 queued

Observability

Programs that watch other programs without disturbing them.

`prof` Sampling CPU profiler

20 commits · 752 lines · 256 test lines

`prof` → A planted hot loop with known 10:1 ratios

HARD INVARIANT	Profiling does not change the profiled program
RESULT	`15 passed, 0 failed, 0 skipped`
PLAN	18 done, 12 queued

`trace` ptrace syscall tracer

5 commits · 481 lines · 443 test lines

`trace` → strace -c and strace -f -c

HARD INVARIANT	Tracing does not change the traced program
RESULT	`23 passed, 0 failed`
PLAN	11 done, 9 queued

Supply chain & process

Programs that check the other programs.

unrepro — Build-reproducibility differ

30 commits · 950 lines · 436 test lines

unrepro → Planted causes, each asserting its diagnosis

HARD INVARIANT	Never report clean on a build that is not reproducible
RESULT	101 passed, 0 failed
PLAN	12 done, 12 queued

commitq — Multi-repo commit gate and GUI

6 commits · 1,002 lines · 326 test lines

commitq → Throwaway repos in a temp directory

HARD INVARIANT	No commit is ever created with a date other than now
RESULT	57 passed, 0 failed
PLAN	5 done, 11 queued

srcdiff — Supply-chain artifact auditor

5 commits · 473 lines · 383 test lines

srcdiff → Real xz upstream artifacts

HARD INVARIANT	No false negatives in HIGH, few enough FPs to stay visible
RESULT	25 passed, 0 failed
PLAN	6 done, 11 queued

What went wrong in the harness

These are the defects worth reading. In each case the code looked fine, the suite was green, and **the thing that was broken was the thing meant to find breakage.**

`inflate` **`make test-asan` could not fail**

Reject tests assert exit code exactly 1; AddressSanitizer also exits 1. Found by deleting the classic `dist > o->n` bounds check and watching 146 tests pass while ASan printed a heap overflow to stderr. Fixing the exit code revealed that every malformed input had also been leaking 192KB, invisible for the same reason.

`walkv` **Half the crash matrix never crashed**

In torn-write mode the `fsync` boundary called the injection hook and discarded its return value, so every fsync scenario ran to completion and reported a perfect result.

`walkv` **Four planted defects, only one caught**

Mutation testing showed the matrix crashed, recovered, verified and then *stopped* — nothing ever wrote after a recovery, so a subtly wrong recovery was unobservable. Adding that one scenario took the suite from 83 to 163 and the defect now fails 20 of them.

`tarx` **Every symlink assertion was vacuous**

`diff -r` dereferences symbolic links, so the comparisons silently compared targets instead of links, and a dangling link aborted the diff outright.

`srcdiff` **Sixteen false positives on a clean release**

Against the genuine xz-5.4.5 tarball and its git tag, every HIGH finding was wrong — and each was an `m4/*.m4` file absent from git, the exact shape of the attack it exists to find. The real signal would have been one line in seventeen.

`dwarfdump` **ASan's exit code collided with “malformed input”**

Both are 1, so the robustness sweeps would have scored a genuine heap overflow as a clean diagnostic.

`unrepro` **Reported REPRODUCIBLE because `make` did nothing**

Pointed at four sibling repos it compared zero outputs and called them clean. Zero outputs is now *inconclusive*.

`commitq` **The commit gate was not gating 12 of 18 repos**

The test command was a hardcoded table of six names; everything else returned success, so two thirds of the project could be committed without running anything.

`slab` **gcc deleted the oracle**

At `-O2` gcc knows what `calloc` promises, so it proved the zero-checking loop could not observe a non-zero byte and removed it. An allocator that never zeroed anything passed cleanly.

`cc` **Five hand-computed expectations were wrong**

Each cost a detour through “is this a codegen bug?” The compiler was right every time. `t/addcase.sh` now takes the expected value from gcc and refuses the case if clang disagrees.

What went wrong in the code

Found by the oracles, never by re-reading the source. Each of these produced plausible output and passed every check that existed at the time.

`rx` **The linear-time guarantee was quietly broken**

Injecting a start thread at every input position collided with the generation marks — and that dedup *is* the guarantee. The one bug that destroyed the project's only interesting property.

`lnk` **A binary the kernel refused to run**

File offset and virtual address were aligned independently. `BASE 0x400000` is divisible by every power of two up to 4MB, so they stayed in step by arithmetic luck until an 8MB-aligned array broke the congruence. The fix made the whole class unemittable rather than merely fixed.

`jail` **A perfect sandbox applied to the wrong process**

The rule-installing loop reused the argument parser's index, so `jail` built a flawless ruleset and then exec'd `--ro`. Every internal invariant held; only a real program reporting a real errno could see it.

`dwarfdump` **Strings resolved to different real strings**

clang emits `DW_AT_name` as `strx1` before declaring `str_offsets_base` on the same DIE. Every lookup succeeded, no error, no marker — the second unit was simply named `main.c` where readelf said `lib.c`.

`trace` **A SIGTERM death reported as exit 0**

After passing a signal through, the tracee was already running, so the next `PTRACE_SYSCALL` hit `ESRCH`. Reintroduced once by a later refactor and caught again immediately by the same test.

`prof` **A race against the dynamic loader**

A one-shot `/proc/PID/maps` snapshot raced `ld.so`, truncating libc frames non-deterministically.

`sqlr` **A closing paren ended the column list**

Depth was decremented and tested in the same iteration, so the `)` of `VARCHAR(10,2)` looked like the end of the CREATE statement. `SELECT *` returned one column of four, with no error.

`srcdiff` **A funding link parsed as a repository**

`github.com/sponsors/hynek` resolved to owner *sponsors*. No offline test could reach it: the bug was in metadata resolution, and the tests all covered the comparison.

HOW THIS IS KEPT HONEST

Every commit builds. Verified by checking out each one in turn, deleting build artifacts so a stale binary cannot fake a pass, and rebuilding.

All eighteen build reproducibly. Proven by `unrepro`, one of the eighteen, which found that eight of its siblings were not — `-g` writes the compiler's working directory into DWARF `.debug_line_str`. Every repo compiling with `-g` was affected and every repo without it was already clean, which is independent confirmation of the cause rather than a plausible guess.

No commit can be gated by a suite that does not run. `commitq` runs each repo's tests against the working tree before committing and refuses when they fail — or when a repo has no discoverable suite at all, which was a real bug it had for two thirds of the project.

75 deliberate shortcuts are marked `ponytail:` in the source, each naming its own ceiling and the upgrade path. 248 planned steps are done and 208 remain, ordered, each recording why it is next.

Generated from the repositories by `.portfolio/build.py`. Data-fill colours validated against the paper surface with the dataviz palette checker: worst CVD ΔE 12.6, normal-vision ΔE 21.0, all six checks pass.